

Obfuscated Secrets (Reverse Engineering)

by MetaCTF

Problem Statement

I hid the flag in this Python script
[https://metaproblems.com/4404b6b97072820c4b3bb702d1846fae/flag_checker.py]. It asks you for a flag and tells you if it's correct or not. Can you reverse engineer it to find the correct flag?

The flag that the script above validates as correct is the flag for this challenge.

You can also access the script in an online Python IDE here: <https://www.online-python.com/1i6lX9bv0a>. Click the green "Run" button to run the program.

Solution

The program takes in a provided flag and compares it against a hardcoded transformed version of the flag (`Lcq]>N?s]bV[R[_0aScQ]]NGPYDKDNG`). We need to reverse the transformation to get the flag.

There are two main checks:

`if len(flag) != len(encrypted): return False` checks if length of entered flag matches the length of the hardcoded string, which is 32 characters

`return all([ord(flag[i]) - 1 == ord(encrypted[i]) + i for i in range(len(encrypted))])` checks if the ascii value of the flag minus 1 minus it's position in the string matches the ascii value of the hardcoded string. This can be easily reversed with a script like this:

```
flag = "Lcq]>N?s]bV[R[_0aScQ]]NGPYDKDNG"

final_flag = ""
for index in range(len(flag)):
    ascii_value_of_character = ord(flag[index])
    transformed_character = chr(ascii_value_of_character + 1 + index)
    final_flag += transformed_character

print(final_flag)
```

Or more simply:

```
print("".join([chr(ord(flag[i]) + 1 + i) for i in range(len(flag))]))
```